ELEC3104: Digital Signal Processing Lab

Professor Eliathamby Ambikairajah & Dr Phu Le

School of Electrical Engineering and Telecommunications

UNSW, Sydney, Australia

Term 3, 2021

Programming in MATLAB

- Matlab Tutorial Videos available at http://eemedia.ee.unsw.edu.au/MatlabTutorial/index.htm
- MATLAB, an abbreviation of MATrix LABoratory, is a proprietary mult- paradigm programming language and numeric computing environment developed by MathWorks.
- MATLAB provides an on-line help using the help command
- For example, to find information about a function plot , type the following at the MATLAB prompt (>>):

  >> help plot

- When unsure about a command , use help
- MATLAB variables are, scalars, vectors, matrices or strings.
- All numerical variables in MATLAB are matrices which is a two-dimensional array of numbers
- MATLAB keeps track of the dimension of each variable. Thus, the following statement creates a scalar variable *x*:

  >> x = 5;

- In MATLAB , a scalar is a variable with one row and one column
- MATLAB variables may also be strings, which are matrices of individual characters
- The following statements assign a numerical value to *x* and a string value to *y*

  >> x = 2.5;
  >> y = 'unsw';

Programming in MATLAB

Scalar operations

```
>> x = 1;
>> y = 2;
>> z = x + y;
```

- If an expression is followed by a semicolon(;), then the result is not displayed on the screen
- If the semicolon is omitted then the result is displayed

```
>> z = x + y
    z =
        3
```

- The following statements demonstrate scalar addition, subtraction, multiplication and division (you may try the following).

```
>> u = 5;        %assign u =5
>> v = 3;        %assign v=3
>> w = u + v     %calculate sum
>> x = u - v
>> y = u * v
>> z = u/v
```

- Note that the expressions w, x, y, z are not followed by a semicolon(;), therefore the results will be displayed on the screen

```
w =
     8
x =
     2
```

and so on.

- % indicates comment(s) in MATLAB

Programming in MATLAB

Matrix operations

- For example, a matrix *M* in linear algebra is defined by

$$M = \begin{bmatrix} 1 & 3 & 5 \\ 6 & 2 & 4 \end{bmatrix}$$

- In MATLAB, a statement

```
>> M = [1 3 5; 6 2 4]
```

creates the above 2 x 3 matrix *M*.

```
M =
     1     3     5
     6     2     4
```

- The size of the matrix can always be extracted using the size operator

```
>> Msiz = size(M)
Msiz =
     2     3
```

- Individual elements of a matrix variable may be accessed by giving the row index and the column index. If *M* is a matrix defined earlier , then *M*(2, 3) is the element in the second row and third column.
- Ask MATLAB to print the (2, 3) element on the screen

```
>> M(2, 3)
ans =
     4
```

- Reassign the (2, 3) element to -2

```
>> M(2, 3) = -2
M =
     1     3     5
     6     2    -2
```

Programming in MATLAB

Vector operations

- In MATLAB a vector is a matrix with either one row or one column
- The distinction between the row vectors and the column vectors is essential
- To create a row vector of length 6, filled with ones use the following:
- >> x = ones(1, 6)

```
x =
     1  1  1  1  1  1
```

- To create a column vector of length 5, filled with zeros use the following:

```
>> x = zeros(5, 1);
```

- Once a vector (*x*) has been created, it may be assigned to another vector (*y*).

```
>> x = zeros(1, 5);
>> y = x;
```

The Colon Operator

- The colon operator (:) is useful for creating index arrays, creating vectors of evenly spaced values and accessing submatrices
- Colon notation can be used to create a vector (**x**) as follows

```
>> x = begin:inc:finish
```

OR

```
>> y = begin:finish
```

Programming in MATLAB

- where *begin* and *finish* are the range of values covered by elements of the *x* vector, and *inc* is the (optional) increment
- If *inc* is omitted, the default increment is 1
- The numbers *begin, inc*, and *finish* need not be integers
- The following statement creates a row vector.

```
>> x = 1:6
    x =
        1  2  3  4  5  6
>>x(end)
     ans =
            6
```

- To create a column vector, append the transpose operator to the expression

```
>> x = (1:3)'
```

```
x =
     1
     2
     3
```

❖ *Using colon notation to create a vector that requires you to specify the increment. Try this example by changing the value of skip*

```
>> begin=1; finish=10; skip=5;
>> inc=(finish-begin)/(skip -1);
>> x= begin:inc:finish
```

- Note: For a 4×3 matrix B, a submatrix C can be extracted with B(2:4,1:2)

```
>> B=[1 2 3; 2 3 4;3 4 5;4 5 6];
>> C = B(2:4,1:2);
```

- B(:) gives a 12×1 column vector that is merely the columns of B concatenated together

Programming in MATLAB

Matrix Multiplication

- Matrix multiplication CD can be carried out only if the two matrices have compatible dimensions. That is the number of columns in C must equal to number of rows in D

- In general, if C is $m \times n$, then D must be $n \times p$ and the product CD would be $m \times p$.

Pointwise Array Operations

- In the pointwise case, if we want to multiply the matrices together element- by - element, they must have exactly same size in both dimensions
- To obtain pointwise multiplication in MATLAB, we use the 'point-star' operator $C.*D$

- We would get the same result if we computed $D.*C$

```
>> D = [2  3 -3; 4 -1  2;  5  0 -2];
>> C = [1  1  0; 1  0  1;  1  0  0];
>> z = D.*C     %pointwise multiplication
z =
     2     3     0
     4     0     2
     5     0     0
```

- In order to access all elements in row 2 of matrix $\boldsymbol{z}$, we use colon as a wild card; i.e $\boldsymbol{z}$(2,:) is the second row
- Use pointwise exponentiation to generate a vector $x = 0.8^n$ where n= 0,1,2,…24.

```
>> x=(0.8).^(0:24)
>> x=(0.8).^(n)   % same as previous line
```

Programming in MATLAB

MATLAB SCRIPTS

- Any expression that can be entered at the MATLAB prompt (>>) can also be stored in a text file with an extension .m (e.g. impulse.m)
- *Here is an example to generate and plot an impulse sequence. The following program is called a m-file*

Program 1

```
 clf    %clear old graph
n = -10 : 20; %Generate a vector from -10 to 20
%Generate the impulse delta
 delta = [zeros(1,10) 1 zeros(1,20)];
%Plot the vector delta at the values specified by n
 stem(n,delta);
 xlabel('Time index n'); ylabel('Amplitude');
 title('Unit sample Sequence')
 axis([-10 20 0 1.1])  % sets scaling for the x - and y
- axes on the plot
```

- If these commands are saved in a file named *impulse.m*, then typing impulse at the command prompt (>>) will run the file.

Exercise:

- ✓ What are the purposes of commands *clf, axis, title, xlabel , ylabel, plot and stem*?
- ✓ Modify program 1 to generate a unit step sequence
- ✓ What is the function of the following program? Analyse the program.

```
>> n= -10:20;
>> u = (n==0);   % n ≠ 0
>> stem(n,u);
```

- ✓ What is the difference between the arithmetic operators ^ and .^?
- ✓ What is function of the following program?

```
>> x = [1 -1 2 -2 1 -1 0];
>> y = sum(x(:));  z = sum(x.*x);
```

Programming in MATLAB

Examples: Drop semicolons if you want to display the values of x and y.

- Variable names can store values and matrices in MATLAB. Try the following:

```
>> x1 = sin(pi/4);
>> y1 = sqrt(1-x1*x1);
```

- Plotting is easy in MATLAB. The basic plot command will plot a vector *y* versus a vector *x*. Try the following

```
>> x = [-1 -1 0 1 2 1];
>> y = x.*x + 3*x;
>> plot(x,y)
```

- MATLAB Indexing: Explain the code below

```
>> jj = 1:3:16
>> kk = 100:-2:86
>> tt = pi*[2:(-1/8):0]
```

- Extracting and inserting numbers in a vector: Explain the following code.

```
xp = [ones(1,5),[2:1:10],zeros(1,3)];
xp(4:8)
length(xp)
xp(2:2:length(xp))
xp(0)    %this would fail. Why?
```

- Vector replacement: Explain the result displayed from the following code.

```
xp(2:5) = pi*(1:4)
```

Exercise:

✓ Write a MATLAB code that will replace the even indexed elements of *xp* with a constant value of -2; *xp vector* is given as below

```
xp = [ones(1,5), [2:1:10], zeros(1,2)];
```

✓ Given x = [-1 1 3 4 5]; what are the values of x(0), x(1), and x(5)?

Programming in MATLAB

Vectorisation

- This is an essential programming skill in MATLAB. Loops can be written in MATLAB, but they are not efficient – loop is unbelievably slow in MATLAB
- Avoid loops and use vector notation instead.
- Consider the following three methods for matrix summation:

➢ Method 1 – double loop is needed

```
M = [1 3 5; 6 2 4];
Nrows = 2; Ncolumns = 3;
total_sum = 0;
for m=1:Nrows
    for n = 1:Ncolumns
        total_sum= total_sum + M(m,n);
    end
end
```

➢ Method 2 – sum each column first

```
total_sum = sum(sum(M));
```

➢ Method 3 – sum all elements

```
total_sum = sum(M(:));
```

- Methods 2 and 3 rely on the built-in function *sum.* The third method is the most efficient one, where the matrix M is converted to a column vector with the colon operator (M(:)) and then *sum* function returns the algebraic sum of all elements of M.

Example: Compute the different values of cosine without a loop and plot the values

```
n = -3:0.5:15;
x = cos(pi*n/4);
plot(n,x), grid on
```

Exercise: What is the purpose of the MATLAB commands *rand* and *randn*? Explain the code:

```
x = randn(1,3)
```

Programming in MATLAB

MATLAB Sound

- By using *sound* command at the MATLAB prompt, you are able to hear the sounds in the MATLAB demo.
- Generate a tone (sinewave) in MATLAB and listen to it with the sound command
- Let the frequency of the tone is 1kHz and the duration should be 2 secs. Save the following lines of code as an M-file (mytone.m) and then run it from the command line

```
duration = 2;
fa = 1000;   % tone frequency (Hz)
fs = 8000;   %sampling frequency
t = 0:(1/fs):duration;
x = sin (2*pi*fa*t);
sound(x, fs);
```

Writing a MATLAB Function

- It is possible to write your own functions and add them to the MATLAB environment. The functions that you create are M-files. Consider the following function to generate a cosine wave:

```
function  x = cosgen (f,duration)
%f = desired frequency
%duration = duration of the wave in secs
% x = output vector
n = 0:1/(40*f):duration;  % gives 40 samples per period
x=cos(2*pi*f*n)
```

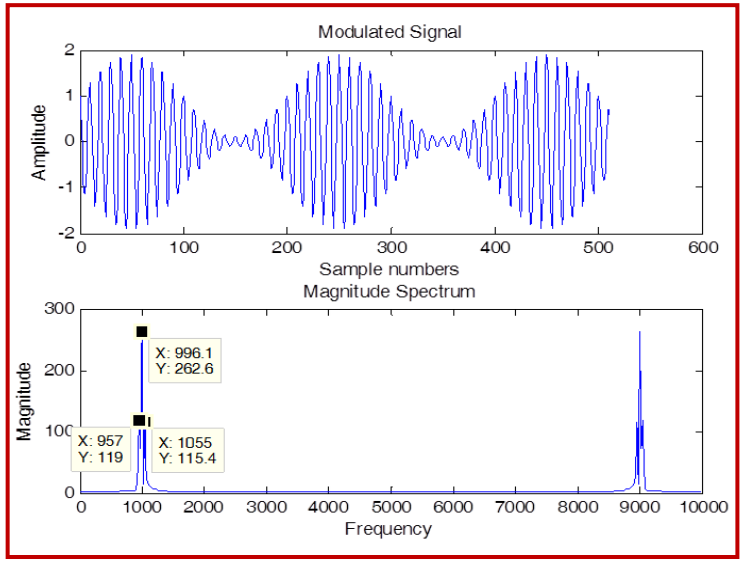
If this file is called cosgen.m, the operation may be invoked from the MATLAB command by typing

```
y = cosgen(1000,2);
```

y will contain the values of the cosine function of frequency 1000Hz and duration of 2 secs

Programming in MATLAB

```
%MATLAB code
Ac=1; m=0.9; fm=50;
fc=1000; fs=10000;
n =0:511;
% x will be a vector of 512 samples
x=Ac*(1+m*sin(2*pi*n*fm/fs)).*cos(2*pi*n*fc/fs);
subplot(2,1,1);
plot(n,x); % Display the signal s[n]
xlabel('Sample number'); ylabel('Amplitude');
title('Modulated Signal');
mag = abs(fft(x)); %Calculate 512-point FFT
res = n*(fs/length(n));
subplot(2,1,2)
plot(res,mag);
xlabel('Frequency'); ylabel('Magnitude');
title('Magnitude Spectrum');
```



Exercise:

- ✓ What are the purpose of *subplot, axis* and *grid* commands?
- ✓ Write a MATLAB program to generate and display a random signal of length 120 samples whose elements are uniformly distributed in the interval [-2,2].
- ✓ Write a MATLAB program to generate and display a Gaussian random signal of length 100 whose elements are normally distributed with zero mean and a variance of 3.

Programming in MATLAB

Exercise:

- ✓ Explain the results obtained from the following MATLAB code:

```
A= randn(6,3);
A = A.* (A>0);
```

- ✓ Explain the following lines of MATLAB code:

```
y = ones(8,1) * rand(1,4);
x = randn(1,3);
 y = x(ones(7,1),:);
```

Complex Numbers: Complex numbers are natural in MATLAB. Try the following:

```
>> z= 3+4i
>> conj(z)
>> abs(z)
>> angle(z)
>> real(z)
>> imag(z)
```

Exercise: Consider the following MATLAB code. What do you expect the plot to look like? Store the MATLAB code in a file called *complex.m* and execute the file from MATLAB prompt.

```
%MATLAB code
fa = 200;
t = [0:1/(20*fa):1];
z = exp(j*2*pi*fa*t);
subplot(2,1,1)
plot(real(z));
title('Real part of z')
subplot(2,1,2)
plot(imag(z));
title('Imaginary part of z')
```

Programming in MATLAB

Example: Write a MATLAB program to solve the following difference equation for $0 \leq n \leq 10$, given that $x[n] = 4\cos(\pi n 0.25)$, $y[0]=1$ and $y[1]=1$.

y[n]=y[n-1]+2*y[n-2]+x[n-2];

```
%MATLAB code
y = [1 1]; % define the first 2 samples of y
x(1)=4;
x(2)= 4*cos(pi/4); %define the first 2 samples of x
for n = 3:11
   x(n)= 4* cos(pi*(n-1)/8); %compute x(n)
   y(n) = y(n-1) +2* y(n-2) +x(n-2); % compute y(n)
end
subplot 211;
stem(x);
xlabel('Time index k'); ylabel ('Amplitude');
subplot 212;
stem(y);
xlabel('Time index k'); ylabel ('Amplitude');
```

Example: A common example of a DSP application is the removal of the noise from a signal corrupted by additive noise e[n]. Let the noisy signal is x[n] = s[n] + e[n] (x[n] is noisy signal, s[n] is clean signal and e[n] is noise). The objective of this example is to remove e[n], by using a three-point moving average algorithm as given below and recover $s[n] \approx y[n]$:

$$y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$$

```
%MATLAB code: signal smoothing algorithm
n=0:0.05:4*pi; R=length(n);%length of the signal x[n]
e=0.8*(rand(R,1)-0.5); %generating noise
s=2*sin(n); % generate clean signal
x=s+e'; %generate noise corrupted signal
y(1)=1/3*x(1);
y(2)=1/3*(x(2)+x(1));%compute the first 2 output samples
for  k=3:R
     y(k)=1/3*(x(k)+x(k-1)+x(k-2)); %3-point averaging
end
subplot 311; plot(s); title('clean signal');
subplot 312; plot(x); title('noisy signal');
subplot 313; plot(y); title('recovered signal');
```

MATLAB Matrices

Exercise:

a. Find a *short* MATLAB expression to build the following matrix:

$$A = \begin{bmatrix} 3 & 4 & 5 & 6 & 7 & 8 \\ 8 & 6 & 4 & 2 & 0 & -2 \\ 1 & 2 & 4 & 8 & 16 & 32 \end{bmatrix}$$

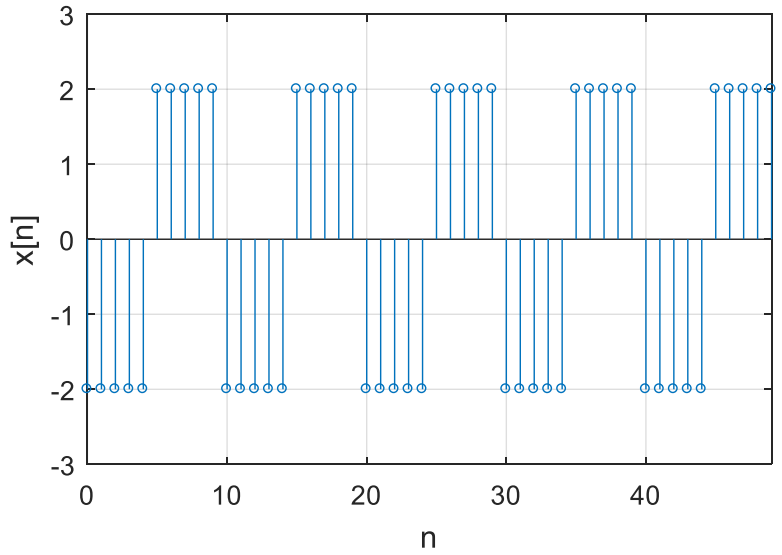
b. Find a MATLAB expression for each of the following:
   i. Obtain the last two rows of A
   ii. Obtain the last three columns of A
   iii. Obtain the even column of A
   iv. Obtain the element in rows 2,3 and columns 4,5,6 of A
   v. Compute the sum of each column of A
   vi. Compute the sum of each row of A

Exercise: In MATLAB

a. Create variables $A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$ and $B = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$
b. Find the difference between two operators A*B and A.*B
c. Assign C = [A B] and D = [A; B]. What is the difference between C and D?
d. Display C, D, size(C), size(D) to verify the result

MATLAB Basic – Signal Generation

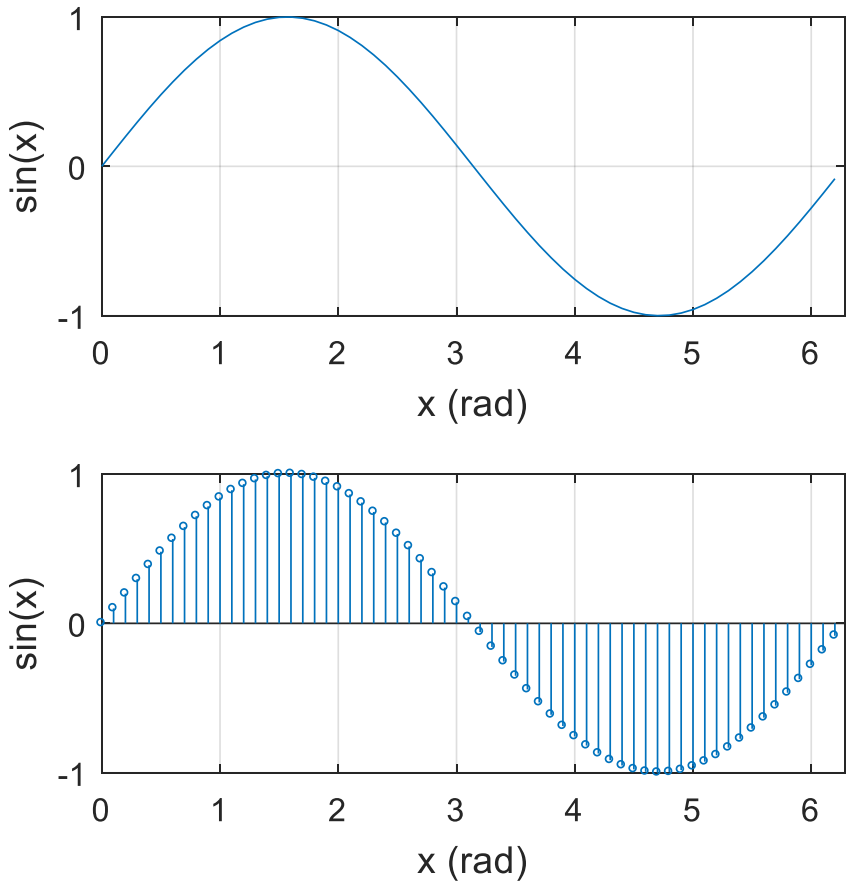
Exercise: Write Matlab code to generate the following signal x[n]



MATLAB Basic – Signal generation & plot

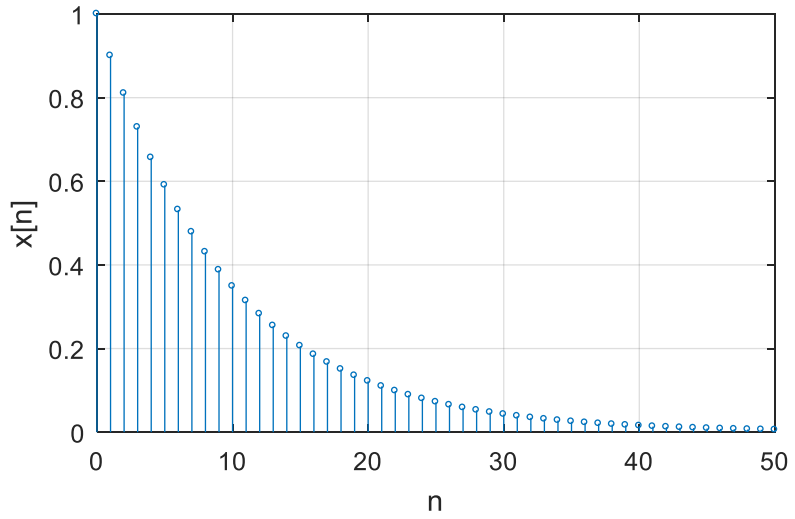
Example: Generate and plot one period of the sinusoid signal

```
x = 0:0.1:2*pi;
y = sin(x);
subplot 211
plot(x,y)
axis([0, 2*pi, -1 1])
ylabel('sin(x)')
xlabel('x (rad)')
grid on
subplot 212
stem(x,y)
axis([0, 2*pi, -1 1])
xlabel('x (rad)')
ylabel('sin(x)')
grid on
```

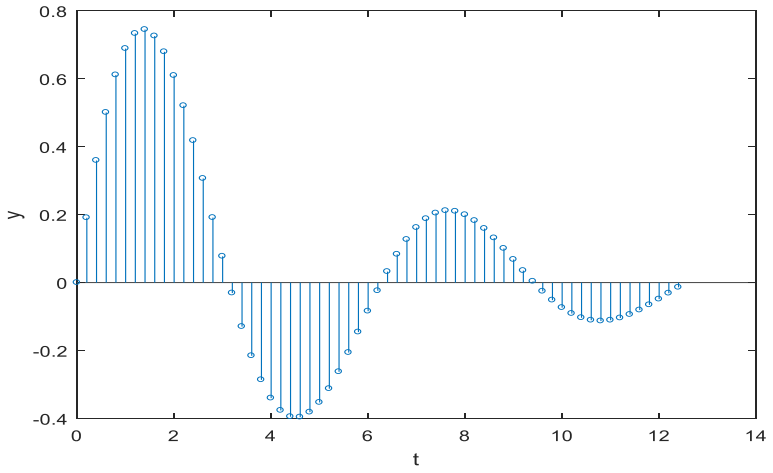


MATLAB Basic – Signal generation & plot (2)

Exercise: Write a Matlab script to generate and plot signal $x[n]=0.9^n$ when $0 \leq n \leq 50$. Your plot should be same as the figure below



Exercise: Write a Matlab script to generate y[t] and plot the signal $y[t]=e^{-0.2t}\sin(t)$ with $t=0, 0.2, 0.4, 0.6, \ldots\ldots, 4\pi$



Topic 1 – Time Domain Analysis

Exercise: Given a *x*[*n*] by the equation:

$$x[n] = sin(\frac{2\pi n f_a}{f_s}),$$

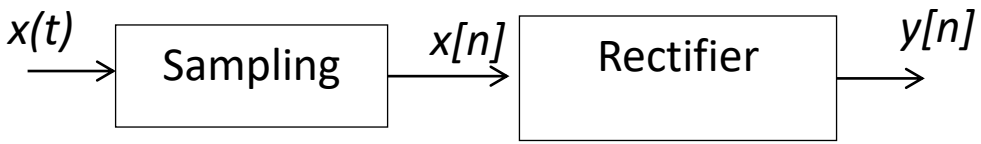
where $f_a = 100Hz$, $f_s = 1\text{k}Hz$, $n = 0,1,2,\dots,24$.

a. Write a MATLAB script to generate and plot x[n].
b. Count the number of samples in one period of x[n] and verify it from the equation given above

Exercise: Consider the discrete-time sequence x[n] = sin (0.25nπ) +sin (0.125nπ+pi/4).

a. Write a MATLAB script to generate 128-samples of *x*[*n*] and plot it.
b. Count the number of samples in one period of x[n] and verify it from the equation given above

Exercise : An analogue signal *x(t) = cos(1000πt)+sin(3000πt)* is sampled at fs=7kHz to obtain x[n] which is then rectified by assigning its negative samples to zero



Using MATLAB to

a. Generate and plot 100ms of signal x[n] with time scale in ms
b. Generate and plot 100ms of signal y[n] with time scale in ms

Topic 1 – Frequency Domain Analysis

Example: In MATLAB:

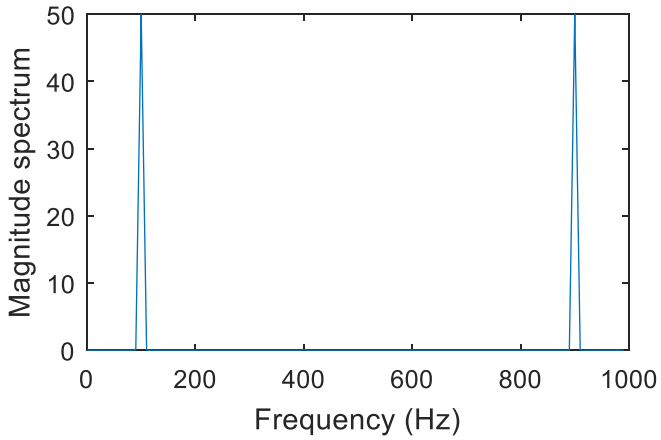
Generate 1000-sample signal $x[n] = sin(\frac{2\pi n f_a}{f_s})$,

where $f_a = 100Hz, f_s = 1\text{k}Hz, n = 0\ to\ 99$.

Compute and plot the magnitude spectrum of x[n] to verify that its frequency is 100Hz.

```
fa=100;
fs=10^3;
n=0:99;
x=sin(2*pi*n*fa/fs);
NFFT = length(n);
x_fft = fft(x, NFFT);
freqHz = (0:NFFT-1)/NFFT*fs

figure
plot(freqHz, abs(x_fft))
xlabel('Frequency (Hz)')
ylabel('Magnitude spectrum')
```



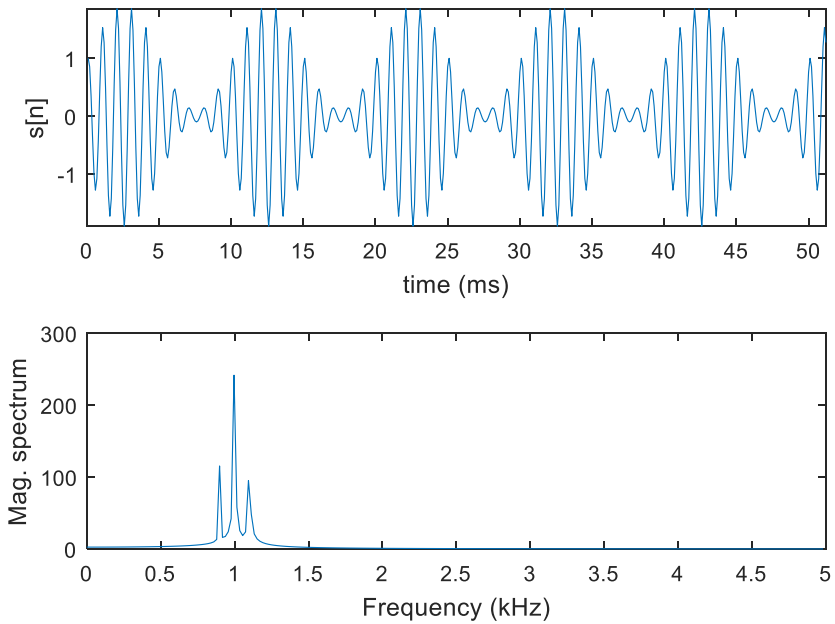
Topic 1 – Frequency Domain Analysis

Exercise: An amplitude modulated signal s[n] is given by,

s[n]=[1+$msin(2\pi nf_m/f_s)$]cos($2\pi nf_c/f_s$), where m = 0.9, f_m=100Hz, f_c=1kHz, f_s=10kHz, n=0,1,2,…,511.

a. Write a MATLAB program to generate and plot s[n].

b. Plot the magnitude spectrum of s[n] in the frequency region [0, $f_s/2$] Hz and identify the frequencies present in the signal.



Topic 1 – Time Frequency Domain Analysis

Exercise: Time-varying stepped frequency

A simple example of time-varying frequency content is to make a waveform whose frequency holds for a short duration and then steps to a higher frequency. For example , C-major scale consists of playing the notes [C(262Hz), D(294Hz), E(330Hz), F(349Hz), G(392Hz), A(440Hz), B(494Hz), C(524Hz)]. In MATLAB:

- ✓ Write a script to generate the signal with the frequency 262Hz for 200ms, then the frequency 294Hz during the next 200ms and so on. The total waveform duration consisting the notes {C,D,E,F,G,A,B,C} will be 1.6 sec. You may choose the sampling frequency as 8000Hz.
- ✓ Play the signal that you have generated using the 'sound' command
- ✓ Plot the spectrogram of the signal that you have generated

References: McClellan J., Schafer R., and Yoder, M., "DSP First – A multimedia Approach", Prentice Hall, 1998

Topic 1 – Sampling Theorem and Aliasing

Exercise: An analogue signal $x(t) = 5\sin(4000\pi t+\pi/4)$ is sampled at a rate of f_s as the following figure to obtain the discrete-time signal x[n]

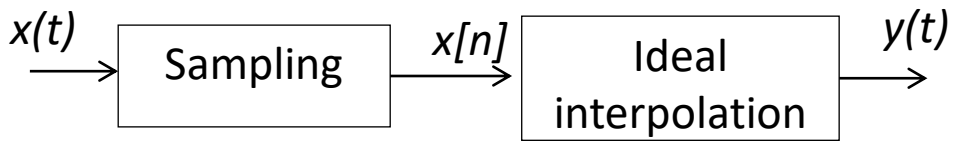


A. What is the minimum sampling frequency of f_s such that the sampling process does not cause any aliasing effect?

B. In MATLAB, generate 1000 samples of signal *x[n]*. Compute and plot the magnitude spectrum of *x[n]* in the frequency region from 0 to f_s
   i. f_s=7kHz, demonstrate that the sampling process does not cause aliasing
   ii. f_s=3kHz, demonstrate that the sampling process causes aliasing

Topic 1 – Sampling Theorem and Aliasing (2)

Exercise : An analogue signal $x(t) = 3\cos(4000\pi t)+5\sin(6000\pi t) +10\cos(12000\pi t)$ is sampled at a rate of f_s as the following figure to obtain the discrete-time signal x[n]



a. What is the minimum sampling frequency of f_s such that the sampling process does not cause an aliasing effect?
b. What is the analogue signal y(t) that can be reconstructed from x[n] if the ideal interpolation is used in the following cases:
   i. f_s=15kHz
   ii. f_s=10kHz
   iii. f_s=7kHz
   iv. f_s=5kHz
c. Verify your answers in (b) by generating a 1000-sample x[n]. Then compute and plot the magnitude spectrum of x[n] versus Hz scale and estimate the frequency components of x[n] in the region [0, f_s/2]